As the default (`standard`) analyzer only works on non-letter delimiters, we need to configure the analyzer to provide any other required patterns. *Patterns* are Java regular expressions provided as a string when configuring the analyzer.

> **NOTE** To learn more about Java regular expressions, go to http://mng.bz/QPDe.

Let's say we have an application that authorizes e-commerce payments, and we receive payment authorization requests from various parties. A 16-digit card number is provided in the format 1234-5678-9000-0000. We want to tokenize this card data on a dash (-) and extract the four tokens individually. We can create a pattern that splits the field into tokens based on the dash delimiter.

To configure the `pattern` analyzer, we must create an index by setting `pattern_analyzer` as the analyzer in the `settings` object. The following listing shows the configuration.

Listing 7.16 Pattern that delimits tokens based on a dash

```
PUT index_with_dash_pattern_analyzer
{
  "settings": {
    "analysis": {
      "analyzer": {
        "pattern_analyzer": {
          "type": "pattern",
          "pattern": "[-]",
          "lowercase": true
        }
      }
    }
  }
}
```

Creates an index with the analyzer settings

In settings, defines the analyzer in the analysis object

Specifies the type of analyzer as pattern

Regex representing the dash

Attaches a lowercase token filter

In the code, we create an index with `pattern` analyzer settings. The `pattern` attribute indicates the regex, which follows Java's regular expression syntax. In this case, we set the dash as our delimiter, so the text is tokenized when it encounters that character. Now that we have the index, let's put this analyzer into action.

Listing 7.17 Testing the `pattern` analyzer

```
POST index_with_dash_pattern_analyzer/_analyze
{
  "text": "1234-5678-9000-0000",
  "analyzer": "pattern_analyzer"
}
```

The output of this command contains four tokens: `["1234","5678","9000","0000"]`.

Text can be tokenized based on a plethora of patterns. I suggest experimenting with regex patterns to get full benefit from the `pattern` analyzer.